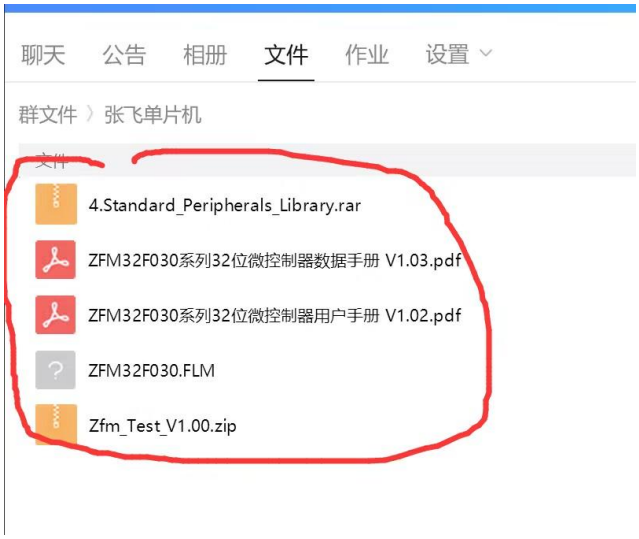
78-张飞单片机

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

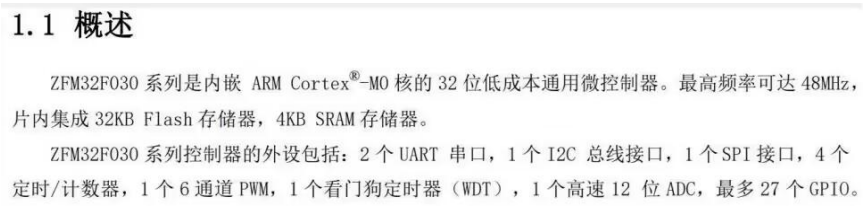
问：群里的代码有啥区别？
答：注意看每个项目内有工程说明文件。

下面，我们来讲下张飞单片机相关的内容。那么，要使用张飞单片机来实现 FOC，首先呢，我们需要了解一下单片机的资源情况。咱们目前呢，使用的是 ZFM32F030K6 这款芯片。

问：这颗张飞的 CPU 带 DSP 引擎么？PWM 可以做到多少评率？中断时间能做到周期的多少呢？？
答：不带，普通的 M0，ARM 单片机，主频 48MHz,中断程序时间 40 多 us,资源紧张，价格低。



张飞单片机开发相关的一些文档，烧写文件在群文件中，可以下载。调试的时候，可以先将电流设定小一些，最好使用带有过流保护的电源来测试，设定好合理的过流值，这样可以最大程度地避免烧板。



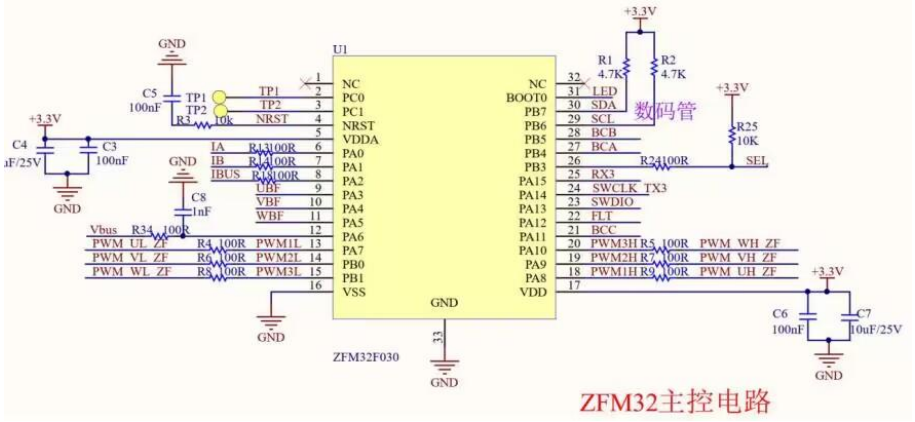
1.1 概述

ZFM32F030 系列是内嵌 ARM Cortex®-M0 核的 32 位低成本通用微控制器。最高频率可达 48MHz，片内集成 32KB Flash 存储器，4KB SRAM 存储器。

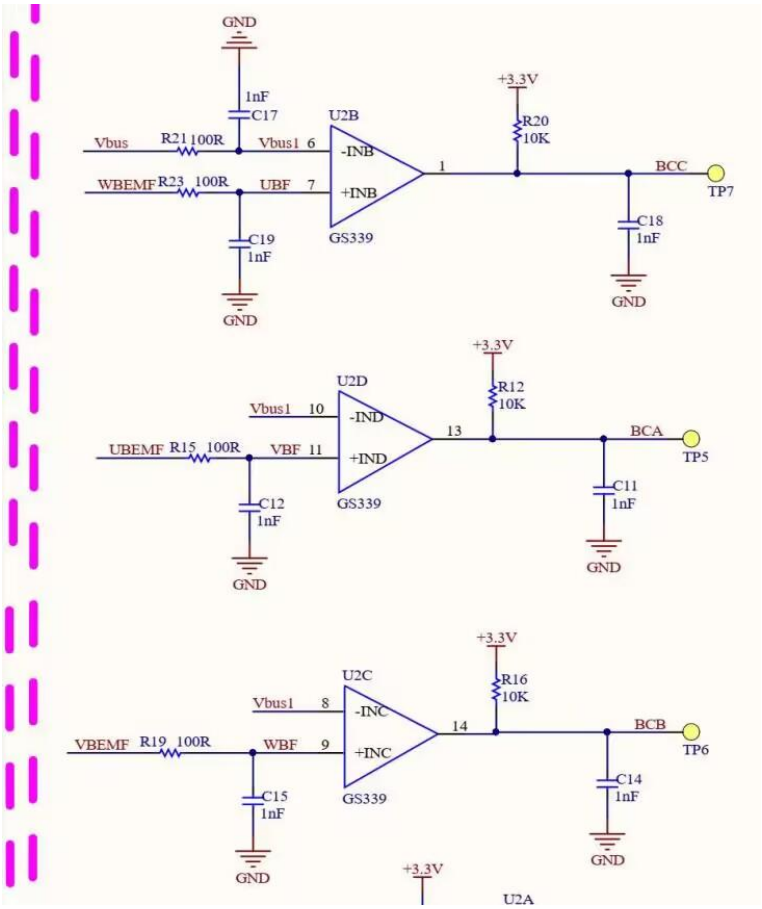
ZFM32F030 系列控制器的外设包括：2 个 UART 串口，1 个 I2C 总线接口，1 个 SPI 接口，4 个定时/计数器，1 个 6 通道 PWM，1 个看门狗定时器（WDT），1 个高速 12 位 ADC，最多 27 个 GPIO。

打开数据手册，大家会看到这样一个概述。介绍了，ZFM32F030K6 的内核，主频、存储器大小及外设资源的情况，内核是 ARM Cortex-M0。如果大家之前用过 ARM 单片机的话，这个上手应该挺快的。咱们呢，主要用到的还是单片机的两个外设，ADC 跟 PWM，其它的都是作为辅助外设使用。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



大家看下，这是 ZFM32 的主控电路，这里，我们仅采样了两路相电流，实现的是双电阻的 FOC。

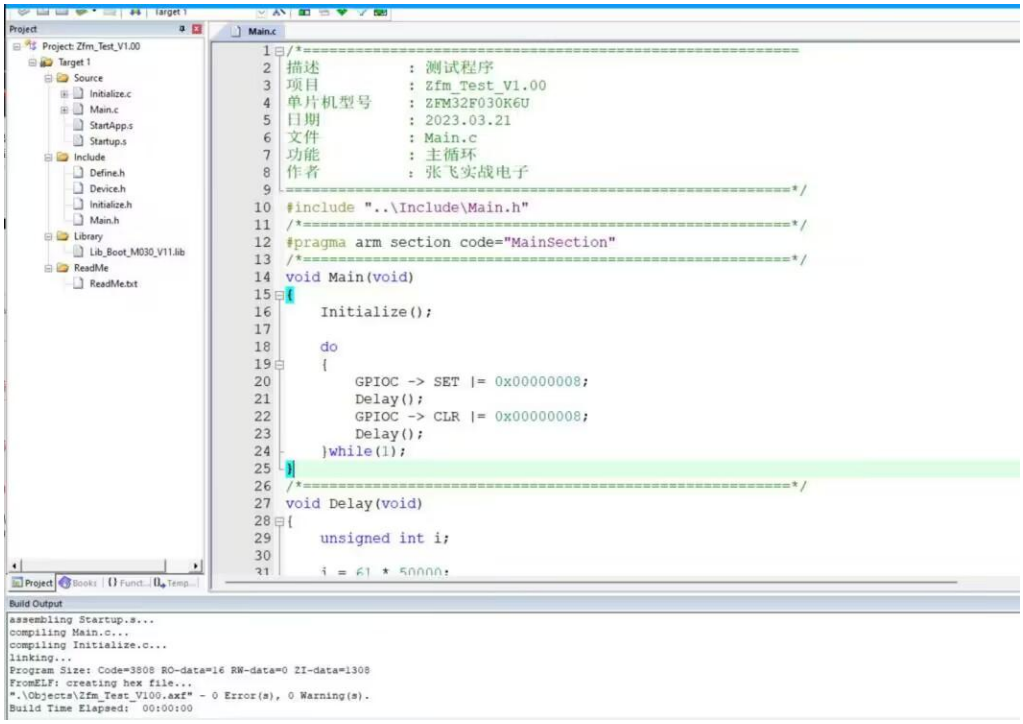


然后，硬件电路上呢，支持无感方波 BLDC，无感方波呢，支持 ADC 过零点检测跟比较器过零点检测。这个是硬件支持的扩展功能，大家如果有时间或者有兴趣，可以编写对应的控制算法。而 ZFM32 的无感 FOC 部分呢，目前是给大家提供库文件的。所以呢，这个部分，我们的目的，是了解一下这款单片机，并且知道如何调用这个库文件，来把电机转动起来。

我们先来了解一下工程架构及 GPIO 端口模块的设置。后面呢，再了解一下 ADC 模块跟 PWM 模块。最后呢，再介绍如何使用库文件。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：什么是库文件呢，这个是直接调用的函数库么？
答：是的，被封装的函数。



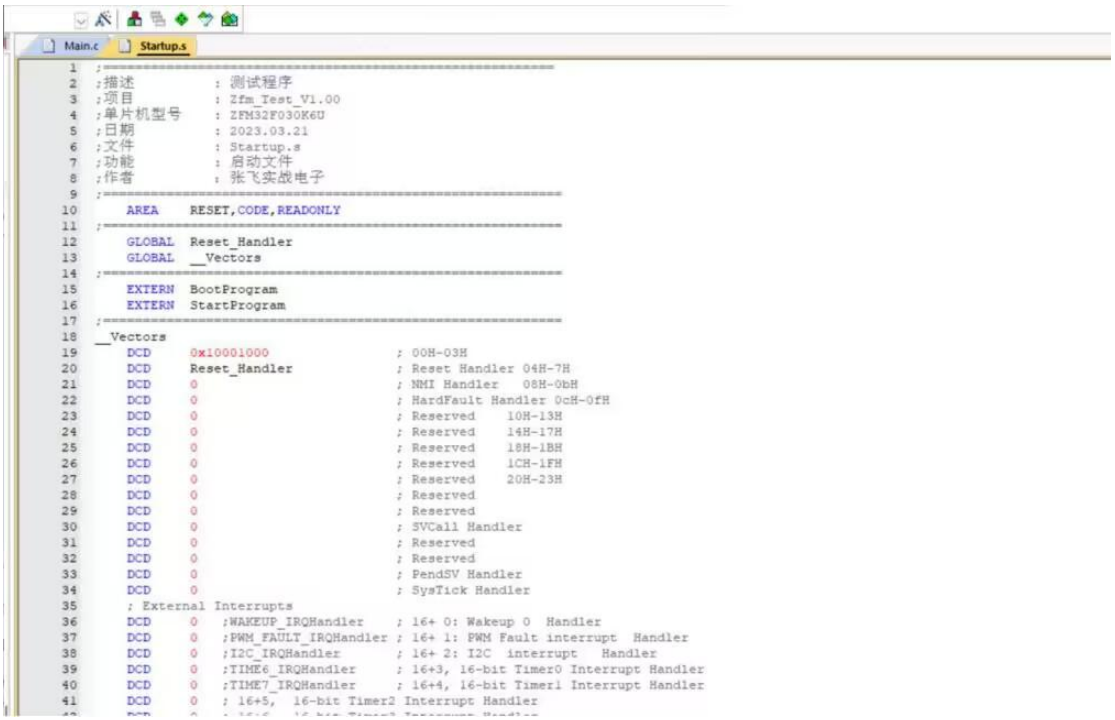


这个代码就在绿框的文件里。这个呢，算是使用我们单片机最简单的模块功能了，仅仅调用了 GPIO 模块。先把最简单的 IO 口了解一下怎么操作。那么，单片机启动的话，都是从启动文件开始运行的。

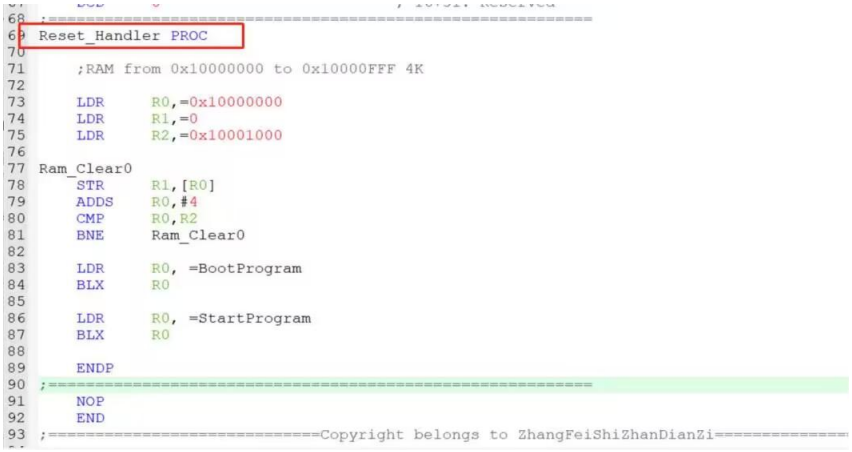
一般呢，ARM 单片机都会提供每款 MCU 对应的启动文件。我们呢，提供了两种启动文件。一种是参照 ARM 风格来写的，这个大家打开标准外设库开发包，可以找到。而另一种呢，是像开发 MICROCHIP 单片机一样，我们自己编写的启动文件。咱们代码中呢，就是使用我们自己编写的启动文件了。

问：还有芯片上电以后怎么找到启动文件.S 的？
答：每个单片机上电以后有复位入口地址，这个地址一般就在中断向量表中。

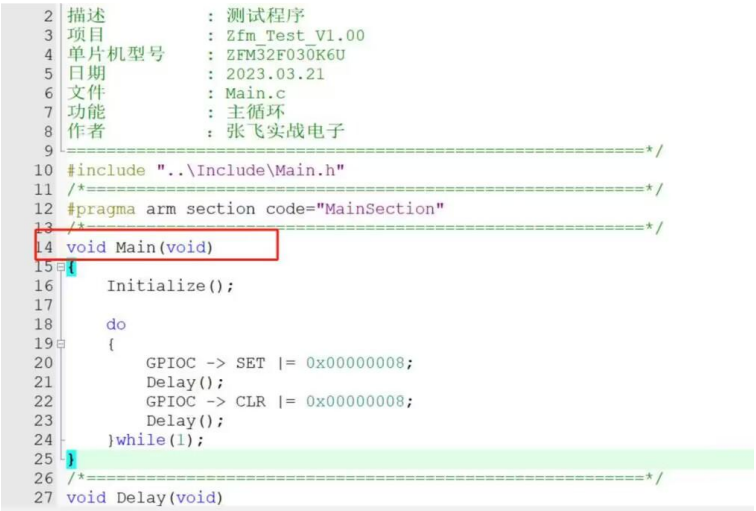
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



就是这个，大致架构，也跟 ARM 提供的一致。



这是单片机上电后，执行的第一个复位函数。然后，这里代码执行后，会跳转到主函数。



也就是，启动文件执行结束后，就会跳转到这里主函数。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

到这里之后呢，就会执行外设初始化配置。这个初始化函数呢，就是配置我们所使用的外设寄存器，还有我们应用程序的初始化。

```
/*============================================================*/
void Initialize(void)
{
    McuInitialize();
    AppInitialize();
}
/*============================================================*/
static void McuInitialize(void)
{
    GpioInitialize();
}
/*============================================================*/
static void AppInitialize(void)
{

}
```

外设初始化呢，就是放在 McuInitialize 函数中调用的。目前代码呢，只写了一个 GPIO 模块的初始化。那么，单片机的引脚，要么是普通的数字 IO，要么是配置为模拟 IO（某些特定引脚），要么是其它的数字外设复用。而具体选择什么功能，要看原理图的设计，根据原理图，来配置对应的模块。我们单片机呢，对某个引脚的功能配置，都有专门的寄存器来配置。每个引脚，都设置了对应的寄存器，我们可以来看看。

```
static void GpioInitialize(void)
{
    IOCFG -> PIOB[3] = 0x00000220; //PB3 SEL
    GPIOB -> DIR |= 0x00000008;

    GPIOB -> SET |= 0x00000008;

    IOCFG -> PIOC[3] = 0x00000221; //PC3 LED
    //LED
    GPIOC -> DIR |= 0x00000008;
    GPIOC -> OUT |= 0x00000008;

    //TP OUT
    IOCFG -> PIOC[1] = 0x00000220;
    IOCFG -> PIOC[0] = 0x00000220;

    //TP
    GPIOC -> DIR |= 0x00000003;
    GPIOC -> OUT |= 0x00000003;

    //FAULT
    IOCFG -> PIOA[12] = 0x00000000;
}
```

这是 IO 端口模块对应的寄存器代码配置。

```
static void GpioInitialize(void)
{
    IOCFG -> PIOB[3] = 0x00000220; //PB3 SEL
    GPIOB -> DIR |= 0x00000008;

    GPIOB -> SET |= 0x00000008;

    IOCFG -> PIOC[3] = 0x00000221; //PC3 LED
    //LED
    GPIOC -> DIR |= 0x00000008;
    GPIOC -> OUT |= 0x00000008;

    //TP OUT
    IOCFG -> PIOC[1] = 0x00000220;
    IOCFG -> PIOC[0] = 0x00000220;

    //TP
    GPIOC -> DIR |= 0x00000003;
    GPIOC -> OUT |= 0x00000003;

    //FAULT
    IOCFG -> PIOA[12] = 0x00000000;
}
```

大家看下，我圈出来的寄存器就是配置某个引脚的功能的。可以配置为普通的数字 IO 口，也可以配置为其它的外设复用口。然后，DIR 是设定方向的寄存器，可以配置为输入或者输出。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

SET 寄存器的话，是设定 IO 引脚的电平为高电平。OUT 寄存器的话，可以设定 IO 引脚的输出电平是高电平或者低电平。

ZFM32F030 系列

32 位微控制器用户手册

V1.02

对应的寄存器详细描述呢，大家可以看下这个用户手册。

```
//FAULT
IOCFG -> PIOA[12] = 0x00000000;
```

比如上面这句话，配置的含义下面都有解释。

4.7.6.13 PA12 寄存器(IOCFG_PA12)

基地址：0x40044000
偏移地址：0x030
复位值：0x00000230
PA12 寄存器功能位选择

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	IEN	S_MODE	OD	DRV	ADM	SRM	INV	CSE	PUE	PDE	FUNC[2:0]		
			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits31:13：保留
Bit12 IEN：输入控制
写/读：
0: 使能
1: 禁止
Bit11 S_MODE：采样模式
写/读：
0: 绕过输入滤波器
1: 采样 1 个滤波器时钟周期。不足 1 个滤波器时钟的输入脉冲将被抑制
Bit10 OD：开漏模式
写/读：
0: 禁止
1: 已使能开漏模式。
注：这不是真正的开漏模式。输入不能上拉至超过 VDD(IO)
Bit9 DRV：驱动电流
写/读：
0: 选择高电流模式
1: 选择低电流模式
Bit8 ADM：模拟/数字功能选择
写/读：
0: 选择数字功能
1: 选择模拟功能

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

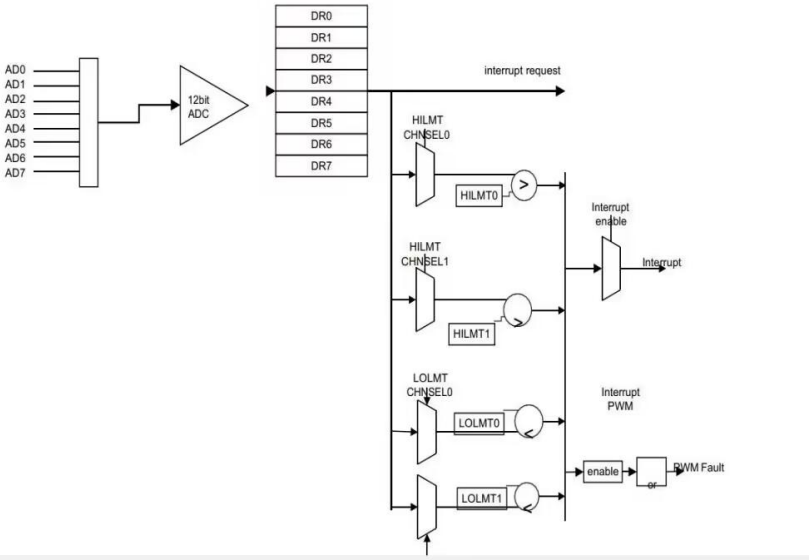
Bit7 SRM：转换速率模式
写/读：
0: 快转换速率模式
1: 慢转换速率模式
Bit6 INV：输入取反
写/读：
0: 无输入取反
1: 输入取反
Bits5 CSE：施密特触发控制
写/读：
0: 使能
1: 禁止
Bit4 PUE：选择管脚模式(上拉电阻控制)
写/读：
0: 无上拉电阻
1: 使能上拉电阻
Bit3 PDE：选择管脚模式(下拉电阻控制)
写/读：
0: 无下拉电阻
1: 使能下拉电阻
Bit2:0 FUNC[2:0]：功能选择
写/读：
0: 选择功能 PA12
1: 保留
2: 保留

这是这个寄存器的描述，全都设置为 0 的话，就是引脚作为一个数字 IO 引脚。因为故障输入的话，就是一个数字输入信号。那么，我们做电机驱动的话，最重要的两个模块就是 AD 模块跟 PWM 模块了。

所以呢，我们也可以了解一下张飞单片机这两个模块的架构。电机驱动，因为要采样特定时刻点的电流，所以 ADC 模块的工作一般是需要跟 PWM 模块进行配合的，一般呢，都是用 PWM 触发 ADC 模块来采样。所以，仅当有触发信号来的时候，ADC 才会开始采样，然后转换出对应的数据。

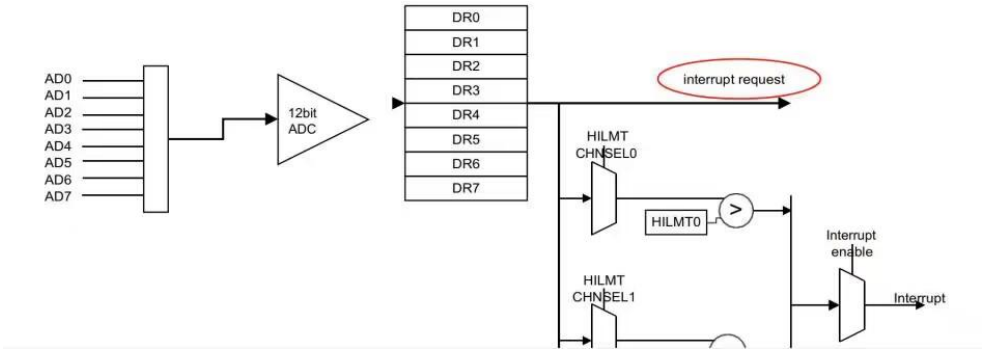
ZFM32F030 系列控制器支持 8 路 12 位的高分辨率的 ADC，主要功能如下：

- 转换速率 1MHz，精度 12 位
- 支持 8 个外部 A/D 通道采样转换
- 支持低功耗模式
- 测量范围为 0~VDDA
- 支持突发模式下的 A/D 转换
- 可配置 ADC 转换触发源：软件触发（Soft trigger），定时器匹配/捕获触发，PWM 重载触发（Reload trigger）
- 每个 ADC 有单独寄存器存储转换结果，从而减少中断负担



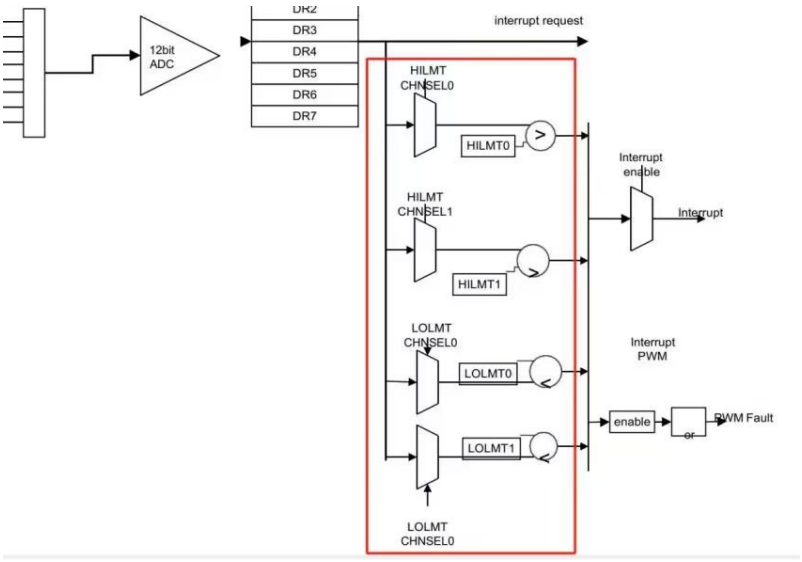
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这幅图呢，可以概括我们单片机的 ADC 模块的功能及特点。主要参数分辨率是 12 位，转换速率是 1MHz。支持 8 路外部的 AD 通道采样。这 8 个通道呢，可以对应着独立的数据存储寄存器。也就是，每个通道配置好之后，数据转换完成，对应的结果就会存储在对应的数据寄存器中。



转换好了之后，支持中断，我们电机控制的算法，就是在这个中断中完成的。也就是，每个 PWM 周期触发一次 ADC 采样，采样转换完成之后，会进入中断，进入中断执行电机控制的程序。

问：一次可以支持几个通道转换？
答：一次触发可以多次多通道采样，最多 8 个通道。



然后，ADC 模块还支持转换的数据阈值比较。如果说转换的结果高于某个阈值，或者低于某个阈值，它可以触发中断，或者作为 PWM 的故障输入信号。那么，要使用这个模块，我们就需要对这个模块的功能寄存器进行配置。

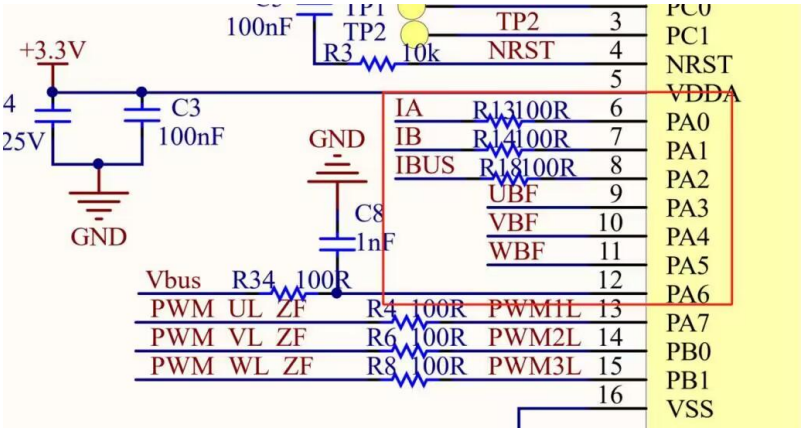
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

13.3.10 ADC 寄存器汇总表

基地址：0x4002 0000

名称	读写	偏移地址	描述
CR	rw	0x000	ADC 控制寄存器。CR 寄存器必须在转换之前进行配置。
GDR	rw	0x004	ADC 全局数据寄存器。包含 ADC转换器最近的转换结果。
CHSEL	rw	0x008	通道选择控制寄存器
INTEN	rw	0x00C	ADC 中断使能寄存器。该寄存器控制 ADC转换结束标志 DONE 是否触发 ADC 中断。
DR0	rw	0x010	A/D 转换结果寄存器 0。该寄存器对应 CHSEL 的 SEL0 定义的通道最近转换值。
DR1	rw	0x014	A/D 转换结果寄存器 1。该寄存器对应 CHSEL 的 SEL1 定义的通道最近转换值。
DR2	rw	0x018	A/D 转换结果寄存器 2。该寄存器对应 CHSEL 的 SEL2 定义的通道最近转换值。
DR3	rw	0x01C	A/D 转换结果寄存器 3。该寄存器对应 CHSEL 的 SEL3 定义的通道最近转换值。
DR4	rw	0x020	A/D 转换结果寄存器 4。.该寄存器对应 CHSEL 的 SEL4 定义的通道最近转换值。
DR5	rw	0x024	A/D 转换结果寄存器 5。该寄存器对应 CHSEL 的 SEL5 定义的通道最近转换值。
DR6	rw	0x028	A/D 转换结果寄存器 6。该寄存器对应 CHSEL 的 SEL6 定义的通道最近转换值。
DR7	rw	0x02C	A/D 转换结果寄存器 7。该寄存器对应 CHSEL 的 SEL7 定义的通道最近转换值。 注意:ADC1DR7 保留为芯片内部温度传感器转换。
STAT	r	0x030	ADC 状态寄存器。该寄存器包含 DONE 和 OVERRUN 标志以及中断标志。
HILMT	rw	0x034	ADC 上限控制寄存器。寄存器控制上限检测功能。
LOLMT	rw	0x038	ADC 下限控制寄存器。寄存器控制下限检测功能。
SSCR	rw	0x040	软件触发转换控制

这是 ADC 模块对应的一些寄存器，我们要使用这个 ADC 模块的话，就要配置这些寄存器，配置为我们需要的功能。



从原理图，我们可以知道，AD 采样使用了 7 个通道。比较重要的模拟通道，我们都放置在了前面。虽然，软件中可以设置通道的转换顺序，但是如果硬件方便连接的话，尽量把重要的信号放在前面。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*============================================================*/
static void AdcInitialize(void)
{
    unsigned int Temp;

    IOCFG -> PIOA[0] = 0x00000102; //IA
    IOCFG -> PIOA[1] = 0x00000102; //IB
    IOCFG -> PIOA[2] = 0x00000105; //IBUS
    IOCFG -> PIOA[3] = 0x00000105; //U Bemf
    IOCFG -> PIOA[4] = 0x00000105; //V Bemf
    IOCFG -> PIOA[5] = 0x00000103; //W Bemf
    IOCFG -> PIOA[6] = 0x00000106; //AVBUS

    SYSCFG -> SYSAHBCLKCTRL |= 0x00004000;
    SYSCFG -> PDRUNCFG &= 0xffffffef;  //ADC Power On
    ADC -> CR |= 0x00000200;
    Temp = 0xff;
    while(Temp -- )
    {
    };
    ADC -> CR |= 0x20000000;
    while((ADC->STAT & 0x00200000) == 0);
    ADC -> CR |= 0x08000000;
    ADC -> CR |= 0x00000047;
    ADC -> CHSEL = 0x76543210;
    ADC -> INTEN |= 0x00000040;
}
```

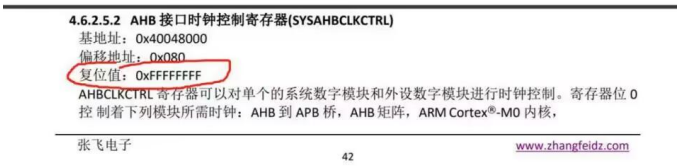
大家可以看下，这是 ADC 模块的配置函数，先是对引脚的功能配置。

```
/*============================================================*/
static void AdcInitialize(void)
{
    IOCFG -> PIOA[0] = 0x00000102; //IA
    IOCFG -> PIOA[1] = 0x00000102; //IB
    IOCFG -> PIOA[2] = 0x00000105; //IBUS
    IOCFG -> PIOA[3] = 0x00000105; //U Bemf
    IOCFG -> PIOA[4] = 0x00000105; //V Bemf
    IOCFG -> PIOA[5] = 0x00000103; //W Bemf
    IOCFG -> PIOA[6] = 0x00000106; //AVBUS

    SYSCFG -> SYSAHBCLKCTRL |= 0x00004000;
    SYSCFG -> PDRUNCFG &= 0xffffffef;  //ADC Power On
    ADC -> CR |= 0x00000200;
    Temp = 0xff;
    while(Temp -- )
    {
    };
    ADC -> CR |= 0x20000000;
    while((ADC->STAT & 0x00200000) == 0);
    ADC -> CR |= 0x08000000;
    ADC -> CR |= 0x00000047;
    ADC -> CHSEL = 0x76543210;
    ADC -> INTEN |= 0x00000040;
}
```

这里是设置 ADC 模块的工作时钟及 ADC 模块的上电功能。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



4.6.2.5.2 AHB 接口时钟控制寄存器(SYSAHBCLKCTRL)

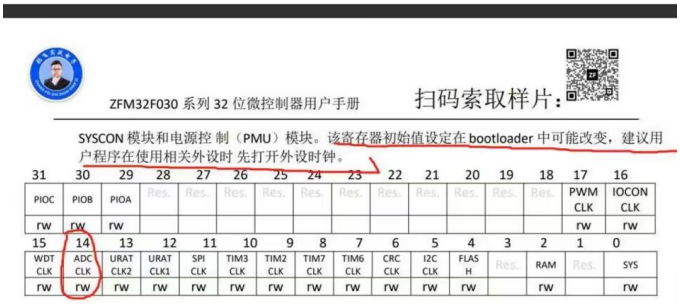
基地址：0x40048000

偏移地址：0x080

复位值：0xFFFFFFFF

AHBCLKCTRL 寄存器可以对单个的系统数字模块和外设数字模块进行时钟控制。寄存器位 0 控 制着下列模块所需时钟：AHB 到 APB 桥，AHB 矩阵，ARM Cortex®-M0 内核，

张飞电子 www.zhangfeidz.com 42



ZFM32F030 系列 32 位微控制器用户手册 扫码索取样片：

SYSCON 模块和电源控 制（PMU）模块。该寄存器初始值设定在 bootloader 中可能改变，建议用户程序在使用相关外设时 先打开外设时钟。

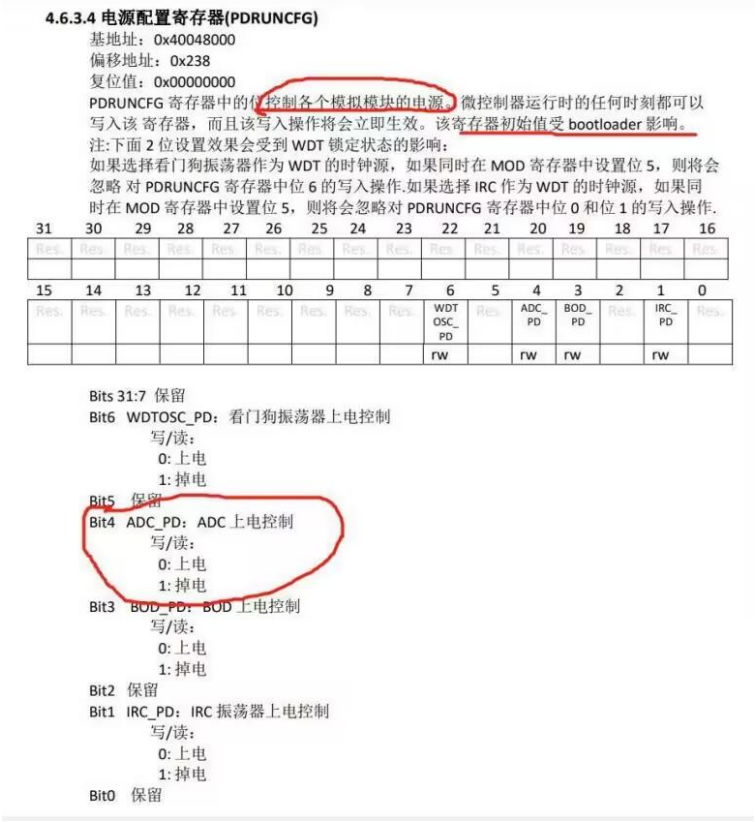
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PIOC	PIOB	PIOA	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PWM CLK	IOCON CLK
rw	rw	rw												rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WDT CLK	ADC CLK	URAT CLK2	URAT CLK1	SPI CLK	TIM3 CLK	TIM2 CLK	TIM7 CLK	TIM6 CLK	CRC CLK	I2C CLK	FLAS H	Res.	RAM	Res.	SYS
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw		rw

大家看，这个时钟控制寄存器，负责单片机外设模块的时钟控制。虽然默认值是打开的，但是也告知我们，可能在 bootloader 中被修改。因此，建议我们使用这个外设模块的时候，再打开一下，所以呢，这里也就需要把 bit14 设置为 1。

Bit14 ADC：ADCCLK 模块时钟控制
写/读：
0: 关闭
1: 打开

ADC 模块时钟 1 是打开，0 是关闭。



4.6.3.4 电源配置寄存器(PDRUNCFG)

基地址：0x40048000

偏移地址：0x238

复位值：0x00000000

PDRUNCFG 寄存器中的位控制各个模拟模块的电源。微控制器运行时的任何时刻都可以写入该 寄存器，而且该写入操作将会立即生效。该寄存器初始值受 bootloader 影响。

注:下面 2 位设置效果会受到 WDT 锁定状态的影响：

如果选择看门狗振荡器作为 WDT 的时钟源，如果同时在 MOD 寄存器中设置位 5，则将会忽略 对 PDRUNCFG 寄存器中位 6 的写入操作.如果选择 IRC 作为 WDT 的时钟源，如果同时在 MOD 寄存器中设置位 5，则将会忽略对 PDRUNCFG 寄存器中位 0 和位 1 的写入操作.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WDT OSC_ PD	Res.	ADC_ PD	BOD_ PD	Res.	IRC_ PD	Res.
									rw		rw	rw		rw	

Bits 31:7 保留

Bit6 WDTOSC_PD：看门狗振荡器上电控制
写/读：
0: 上电
1: 掉电

Bit5 保留

Bit4 ADC_PD：ADC 上电控制
写/读：
0: 上电
1: 掉电

Bit3 BOD_PD：BOD 上电控制
写/读：
0: 上电
1: 掉电

Bit2 保留

Bit1 IRC_PD：IRC 振荡器上电控制
写/读：
0: 上电
1: 掉电

Bit0 保留

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

然后，有一个电源配置的寄存器，控制着 ADC 模块的电源。bit4 就是控制 ADC 模块的上电、掉电，最好呢，我们也是需要设置让它上电，把 bit4 设置为 0 。

```
/*============================================================*/
static void AdcInitialize(void)
{
    IOCFG -> PIOA[0] = 0x00000102; //IA
    IOCFG -> PIOA[1] = 0x00000102; //IB
    IOCFG -> PIOA[2] = 0x00000105; //IBUS
    IOCFG -> PIOA[3] = 0x00000105; //U Bemf
    IOCFG -> PIOA[4] = 0x00000105; //V Bemf
    IOCFG -> PIOA[5] = 0x00000103; //W Bemf
    IOCFG -> PIOA[6] = 0x00000106; //AVBUS

    SYSCFG -> SYSAHBCLKCTRL |= 0x00004000;
    SYSCFG -> PDRUNCFG &= 0xffffffef;  //ADC Power On
    ADC -> CR |= 0x00000200;
    Temp = 0xff;
    while(Temp -- )
    {
    };
    ADC -> CR |= 0x20000000;
    while((ADC->STAT & 0x00200000) == 0);
    ADC -> CR |= 0x08000000;
    ADC -> CR |= 0x00000047;
    ADC -> CHSEL = 0x76543210;
    ADC -> INTEN |= 0x00000040;
}
/*============================================================*/
```

这个地方，是设置 ADC 模块的时钟分频。我们目的呢，是要设置为 16MHz，转换一次 16 个时钟周期的话，就是 1us。所以，这里的分频配置就是 2（3 分频）。

Bits 15:8	CLKDIV[7:0] A/D 转换时钟分频值 ADC 转换时钟由 ADC 外设时钟(PCLK)除 以该分频取得。ADC 转换时钟最大不超过 16MHz。每一次 ADC 转换需要 16 个时钟，ADC 最大采样率为 1MHz。

这个 CR 寄存器的 bit8~bit15，就是设置时钟分频值的。

```
/*============================================================*/
static void AdcInitialize(void)
{
    IOCFG -> PIOA[0] = 0x00000102; //IA
    IOCFG -> PIOA[1] = 0x00000102; //IB
    IOCFG -> PIOA[2] = 0x00000105; //IBUS
    IOCFG -> PIOA[3] = 0x00000105; //U Bemf
    IOCFG -> PIOA[4] = 0x00000105; //V Bemf
    IOCFG -> PIOA[5] = 0x00000103; //W Bemf
    IOCFG -> PIOA[6] = 0x00000106; //AVBUS

    SYSCFG -> SYSAHBCLKCTRL |= 0x00004000;
    SYSCFG -> PDRUNCFG &= 0xffffffef;  //ADC Power On
    ADC -> CR |= 0x00000200;
    Temp = 0xff;
    while(Temp -- )
    {
    };
    ADC -> CR |= 0x20000000;
    while((ADC->STAT & 0x00200000) == 0);
    ADC -> CR |= 0x08000000;
    ADC -> CR |= 0x00000047;
    ADC -> CHSEL = 0x76543210;
    ADC -> INTEN |= 0x00000040;
}
/*============================================================*/
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

接着呢，这里设置一个等待，因为上面是对 ADC 模块上电了，所以，这里需要等待一些时钟周期。

```
static void AdcInitialize(void)
{
    unsigned int Temp;

    IOCFG -> PIOA[0] = 0x00000102; //IA
    IOCFG -> PIOA[1] = 0x00000102; //IB
    IOCFG -> PIOA[2] = 0x00000105; //IBUS
    IOCFG -> PIOA[3] = 0x00000105; //U Bemf
    IOCFG -> PIOA[4] = 0x00000105; //V Bemf
    IOCFG -> PIOA[5] = 0x00000103; //W Bemf
    IOCFG -> PIOA[6] = 0x00000106; //AVBUS

    SYSCFG -> SYSAHBCLKCTRL |= 0x00004000;
    SYSCFG -> PDRUNCFG &= 0xffffffef;  //ADC Power On
    ADC -> CR |= 0x00000200;
    Temp = 0xff;
    while(Temp -- )
    {
    };
    ADC -> CR |= 0x20000000;
    while((ADC->STAT & 0x00200000) == 0);
    ADC -> CR |= 0x08000000;
    ADC -> CR |= 0x00000047;
    ADC -> CHSEL = 0x76543210;
    ADC -> INTEN |= 0x00000040;
}
```

然后，这里是设置外部时钟。也就是把 SCMODE 设置为 1，我们是使用触发模式，必须这样配置。

ADC 控制寄存器用于对A/D 通道选择，采样率，转换模式以及转换条件的设置。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	SC MODE	EDGE	START[3:0]				Res.	Res.	Res.	Res.	Res.	Res.	Res.	BUR ST
		rw	rw	rw	rw	rw	rw								rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CLKDIV[7:0]								CNVEN[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 保留

Bit29: SCMODE ADC 转换采样时钟选择
0: ADC 内部时钟作为采样时钟
1: 外部时钟作为采样时钟，当为触发模式时，必须选为外部时钟模式

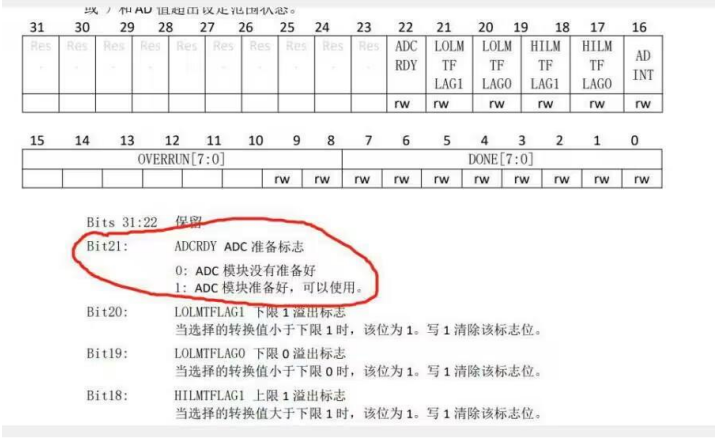
Bit28: EDGE 边沿控制。该边沿控制只对 START 域 3~8 设置有效。
0: 上升沿有效
1: 下降沿有效

Bits 27:24 START[3:0] ADC 转换启动控制。

```
    SYSCFG -> SYSAHBCLKCTRL |= 0x00004000;
    SYSCFG -> PDRUNCFG &= 0xffffffef;  //ADC Power On
    ADC -> CR |= 0x00000200;
    Temp = 0xff;
    while(Temp -- )
    {
    };
    ADC -> CR |= 0x20000000;
    while((ADC->STAT & 0x00200000) == 0);
    ADC -> CR |= 0x08000000;
    ADC -> CR |= 0x00000047;
    ADC -> CHSEL = 0x76543210;
    ADC -> INTEN |= 0x00000040;
}
/*=========================================================
```

接着，需要等待 ADC 准备好。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	Res	Res	Res	Res	Res	Res	Res	Res	ADC RDY	LOLM TF LAG1	LOLM TF LAG0	HILM TF LAG1	HILM TF LAG0	AD INT	
									rw	rw	rw	rw	rw	rw	

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVERRUN[7:0]								DONE[7:0]							
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:22 保留

Bit21: ADCRDY ADC 准备标志

0: ADC 模块没有准备好

1: ADC 模块准备好，可以使用。

Bit20: LOLMTFLAG1 下限 1 溢出标志

当选择的转换值小于下限 1 时，该位为 1。写 1 清除该标志位。

Bit19: LOLMTFLAG0 下限 0 溢出标志

当选择的转换值小于下限 0 时，该位为 1。写 1 清除该标志位。

Bit18: HILMTFLAG1 上限 1 溢出标志

当选择的转换值大于下限 1 时，该位为 1。写 1 清除该标志位。

也就是我们需要等待这个标志位为 1 。

```
Temp = 0xff;
while(Temp -- )
{
};
ADC -> CR |= 0x20000000;
while((ADC->STAT & 0x00200000) == 0);
ADC -> CR |= 0x08000000;
ADC -> CR |= 0x00000047;
ADC -> CHSEL = 0x76543210;
ADC -> INTEN |= 0x00000040;
```

然后，这里是设置触发源，触发源是设置为 PWM 重载事件信号触发。

```
Temp = 0xff;
while(Temp -- )
{
};
ADC -> CR |= 0x20000000;
while((ADC->STAT & 0x00200000) == 0);
ADC -> CR |= 0x08000000;
ADC -> CR |= 0x00000047;
ADC -> CHSEL = 0x76543210;
ADC -> INTEN |= 0x00000040;
```

接着，是使能通道，这个地方，只是采集了电流跟电压，没有采集反电动势。后续，如果要做无感方波，则可以采集端电压。所以，这里就使能了 0 1 2 6。对应三个电流跟一个 Vbus 电压。

问： 既然无感了还不如做成 FOC 吧，硬件是不是几乎是一样的？

答：有区别，各有各的用途，有时候不能相互替代。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    Temp = 0xff;
    while(Temp -- )
    {
    };
    ADC -> CR |= 0x20000000;
    while((ADC->STAT & 0x00200000) == 0);
    ADC -> CR |= 0x08000000;
    ADC -> CR |= 0x00000047;
    ADC -> CHSEL = 0x76543210;
    ADC -> INTEN |= 0x00000040;
}
```

然后，配置 DR0~DR7 对应的 ANx，这里呢，因为硬件上顺序是安排好的，所以，软件中的寄存器设置顺序就是从低往高了。

```
    {
    };
    ADC -> CR |= 0x20000000;
    while((ADC->STAT & 0x00200000) == 0);
    ADC -> CR |= 0x08000000;
    ADC -> CR |= 0x00000047;
    ADC -> CHSEL = 0x76543210;
    ADC -> INTEN |= 0x00000040;
}
/*=========================================================
```

然后，这里就是 ADC 中断 设置的是 通道 6 转换完成触发中断，这样的话，ADC 模块就配置好了。

```
/*=========================================================*
static void AdcInitialize(void)
{
    unsigned int Temp;

    IOCFG -> PIOA[0] = 0x00000102; //IA
    IOCFG -> PIOA[1] = 0x00000102; //IB
    IOCFG -> PIOA[2] = 0x00000105; //IBUS
    IOCFG -> PIOA[3] = 0x00000105; //U Bemf
    IOCFG -> PIOA[4] = 0x00000105; //V Bemf
    IOCFG -> PIOA[5] = 0x00000103; //W Bemf
    IOCFG -> PIOA[6] = 0x00000106; //AVBUS

    SYSCFG -> SYSAHBCLKCTRL |= 0x00004000;
    SYSCFG -> PDRUNCFG &= 0xffffffef;  //ADC Power On
    ADC -> CR |= 0x00000200;
    Temp = 0xff;
    while(Temp -- )
    {
    };
    ADC -> CR |= 0x20000000;
    while((ADC->STAT & 0x00200000) == 0);
    ADC -> CR |= 0x08000000;
    ADC -> CR |= 0x00000047;
    ADC -> CHSEL = 0x76543210;
    ADC -> INTEN |= 0x00000040;
}
```

这就是我们单片机 ADC 模块的寄存器整个初始化设置。

下节课，我们再了解一下 PWM 模块和电机库的调用。
今天内容就到这里，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。